

SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models

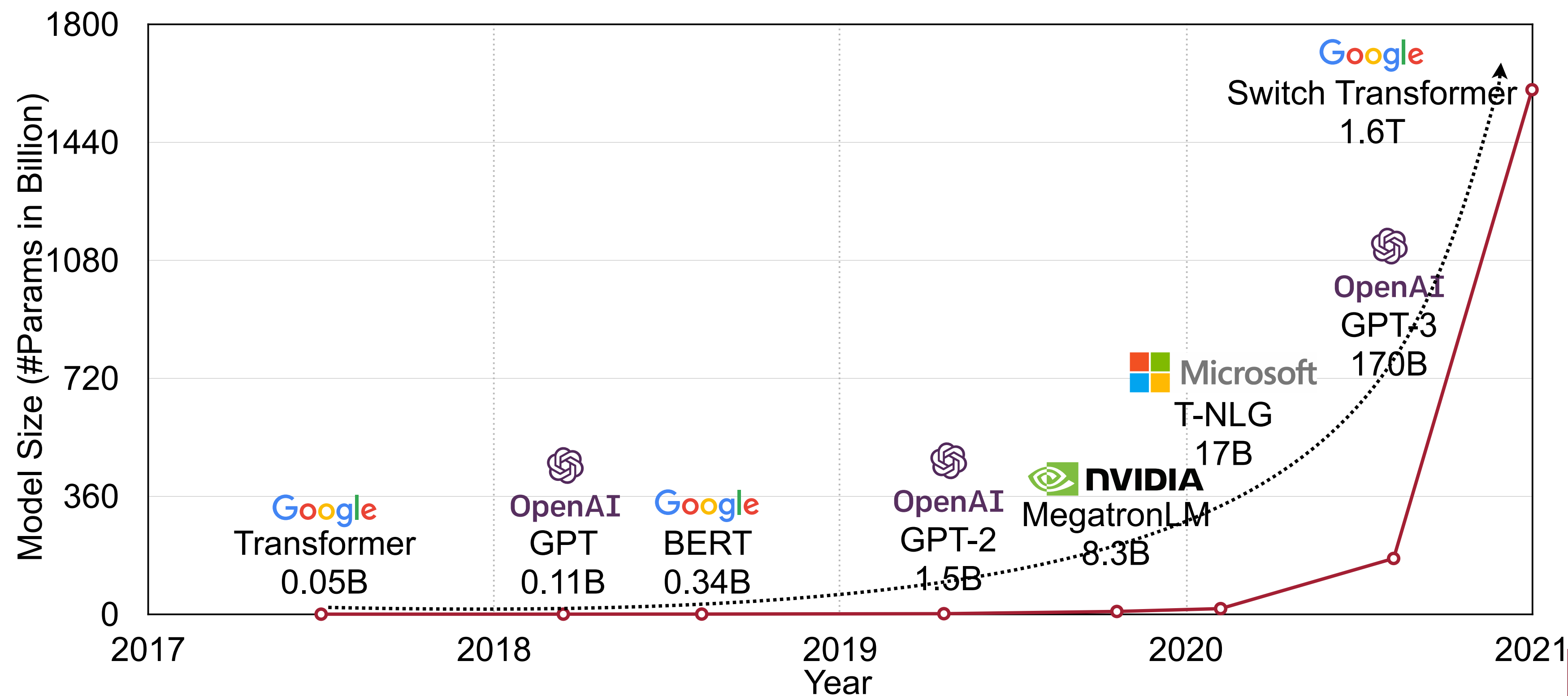
Guangxuan Xiao^{*,1}, Ji Lin^{*,1},
Mickael Seznec², Julien Demouth², Song Han¹

¹ MIT ² NVIDIA

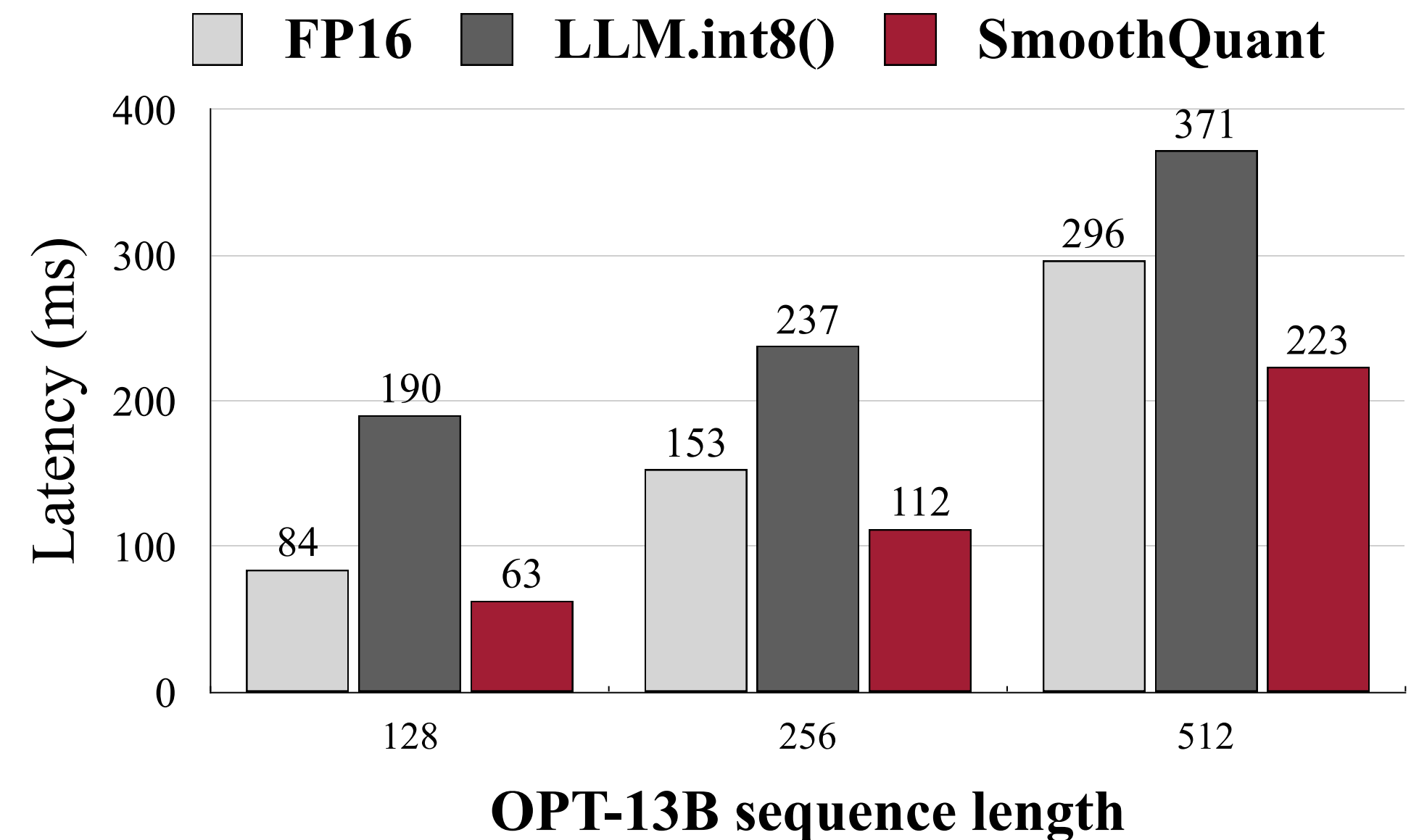
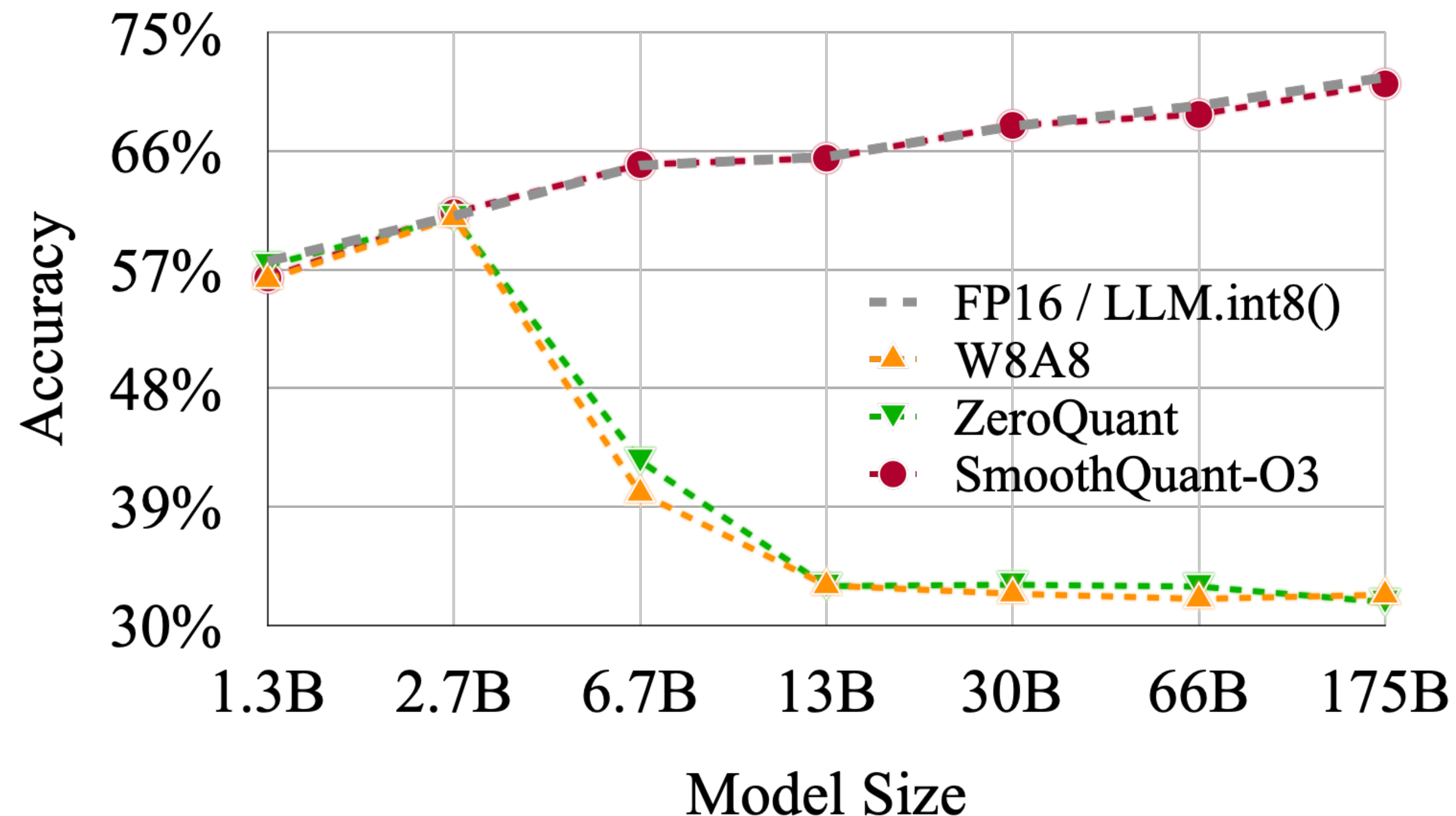
^{*} Equal contribution

Quantization for LLMs is Important

- NLP model size and computation are increasing exponentially. Model Compression with:
 - Quantization (SmoothQuant) $\leq$ today's focus: training-free, model-in & model-out.
 - Token pruning (SpAtten)
 - Neural architecture search (HAT, Lite-Transformer)



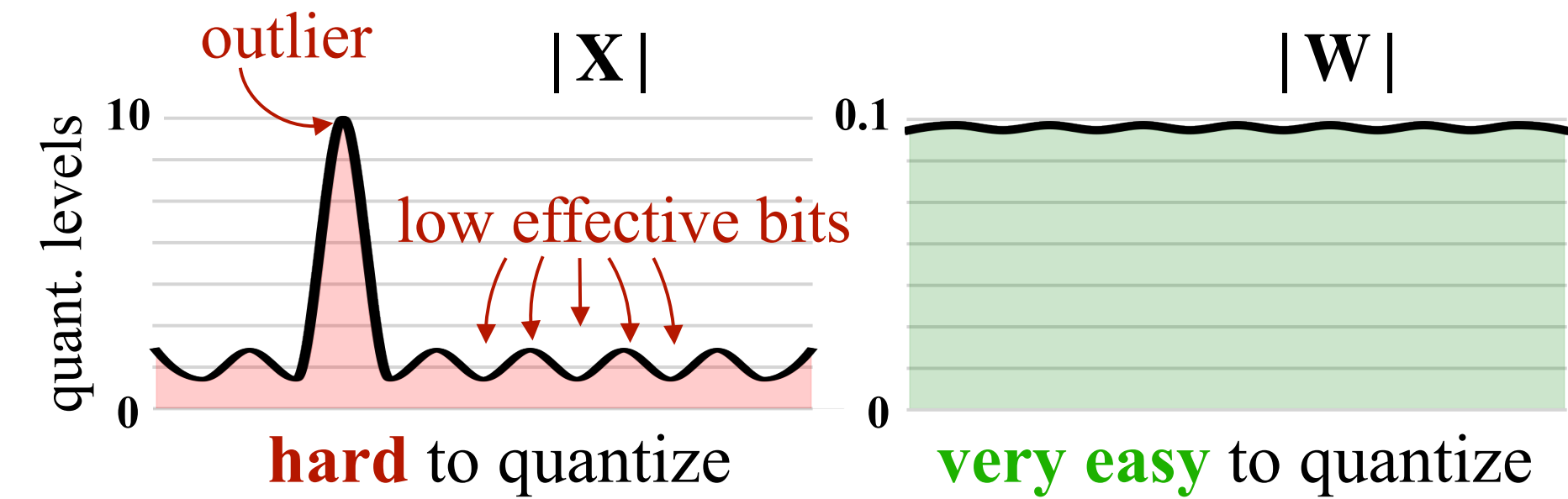
Existing Quantization Method is Slow or Inaccurate



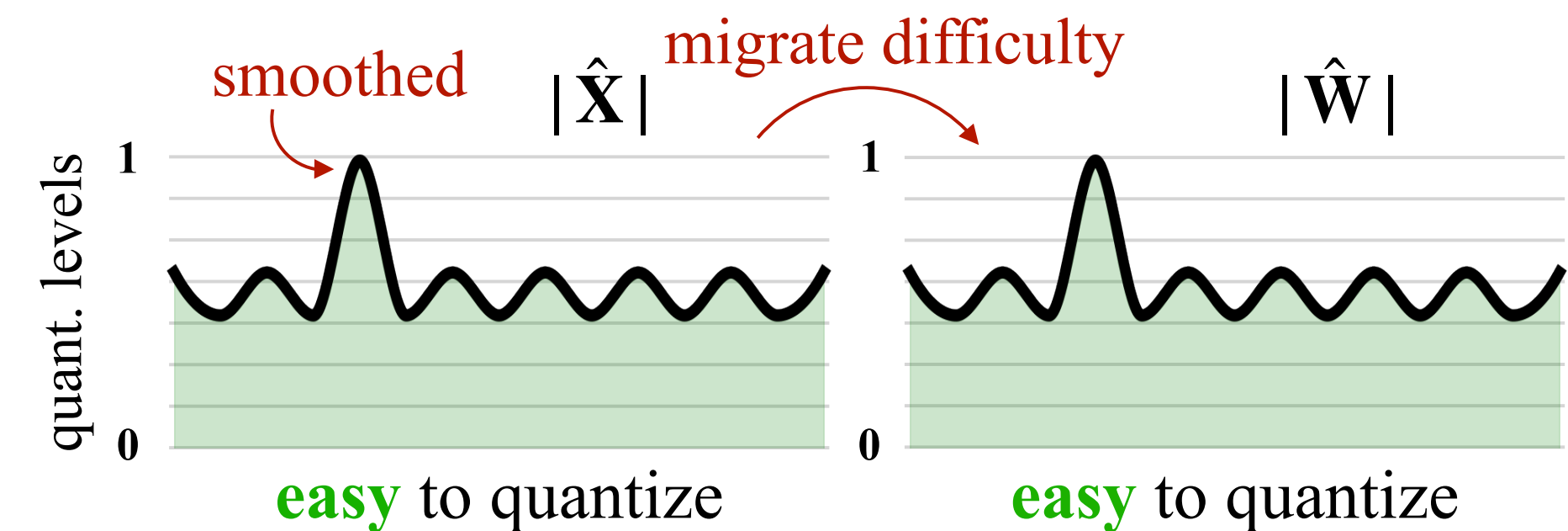
- Systematic outliers emerge in activations when we scale up LLMs beyond 6.7B. Naive but efficient quantization methods will destroy the accuracy.
- The accuracy-preserving baseline, LLM.int8() uses FP16 to represent outliers, which needs runtime outlier detection, scattering and gathering. It is slower than FP16 inference.

SmoothQuant: Accurate and Efficient Post-Training Quantization for LLMs

	LLM (100B+) Accuracy	Hardware Efficiency
ZeroQuant	✗	✓
Outlier Suppression	✗	✓
LLM.int8()	✓	✗
SmoothQuant	✓	✓



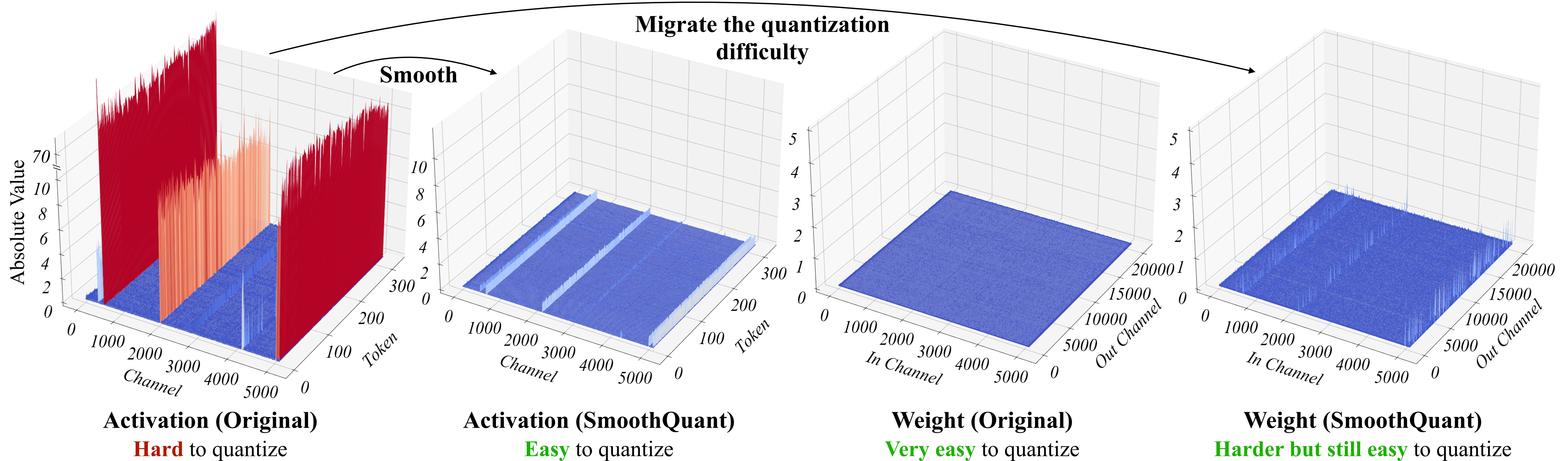
(a) Original



(b) SmoothQuant

- We propose SmoothQuant, an **accurate** and **efficient** post-training-quantization (PTQ) method to enable 8-bit weight, 8-bit activation (**W8A8**) quantization for LLMs.
- Since **weights are easy** to quantize while **activations are not**, SmoothQuant smooths the activation outliers by **migrating the quantization difficulty from activations to weights** with a mathematically equivalent transformation.

Review the Quantization Difficulty of LLMs

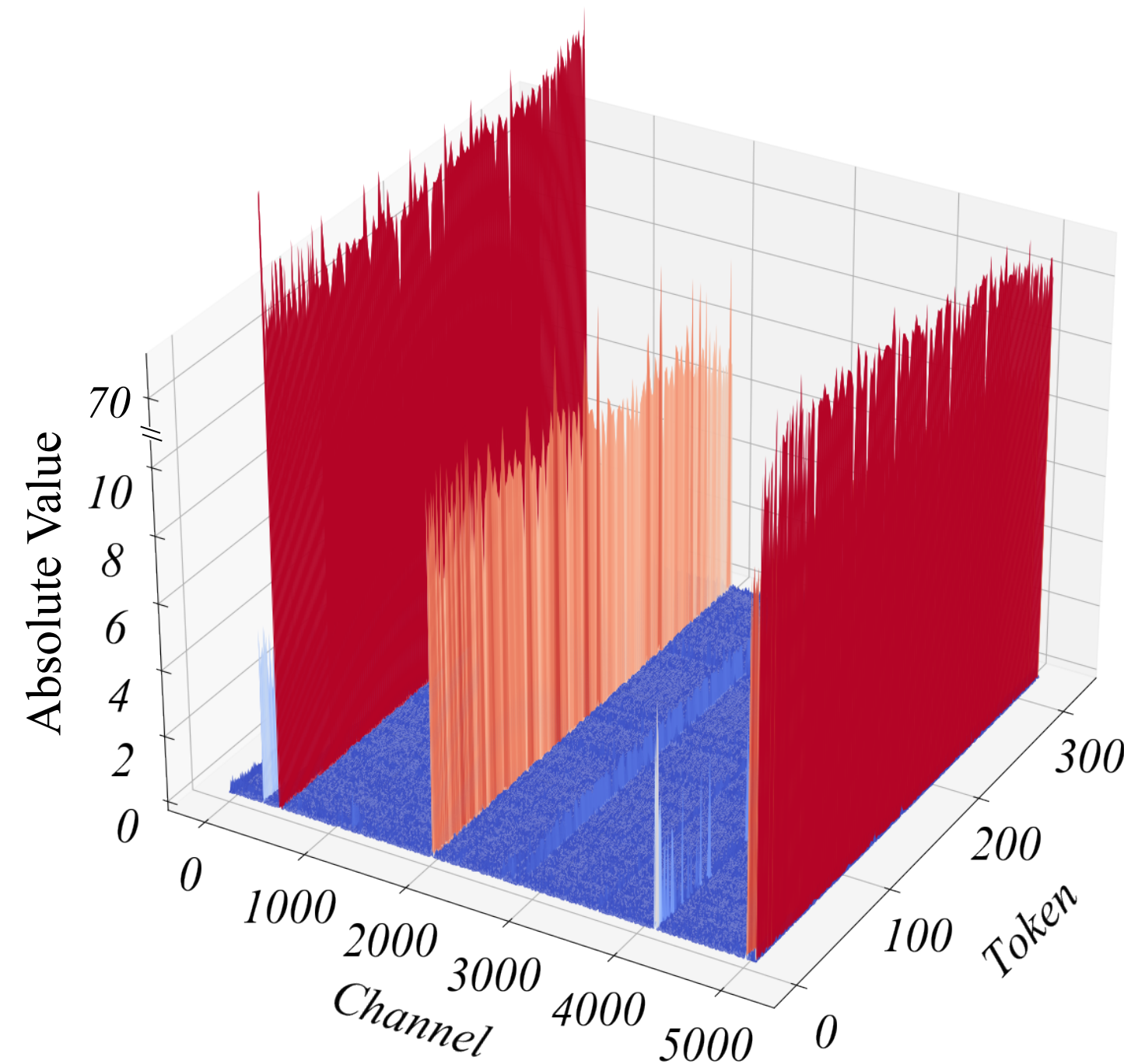


LLMs are notoriously difficult to quantize because:

- Activations are harder to quantize than weights
- Outliers make activation quantization difficult
- Outliers persist in *fixed* channels

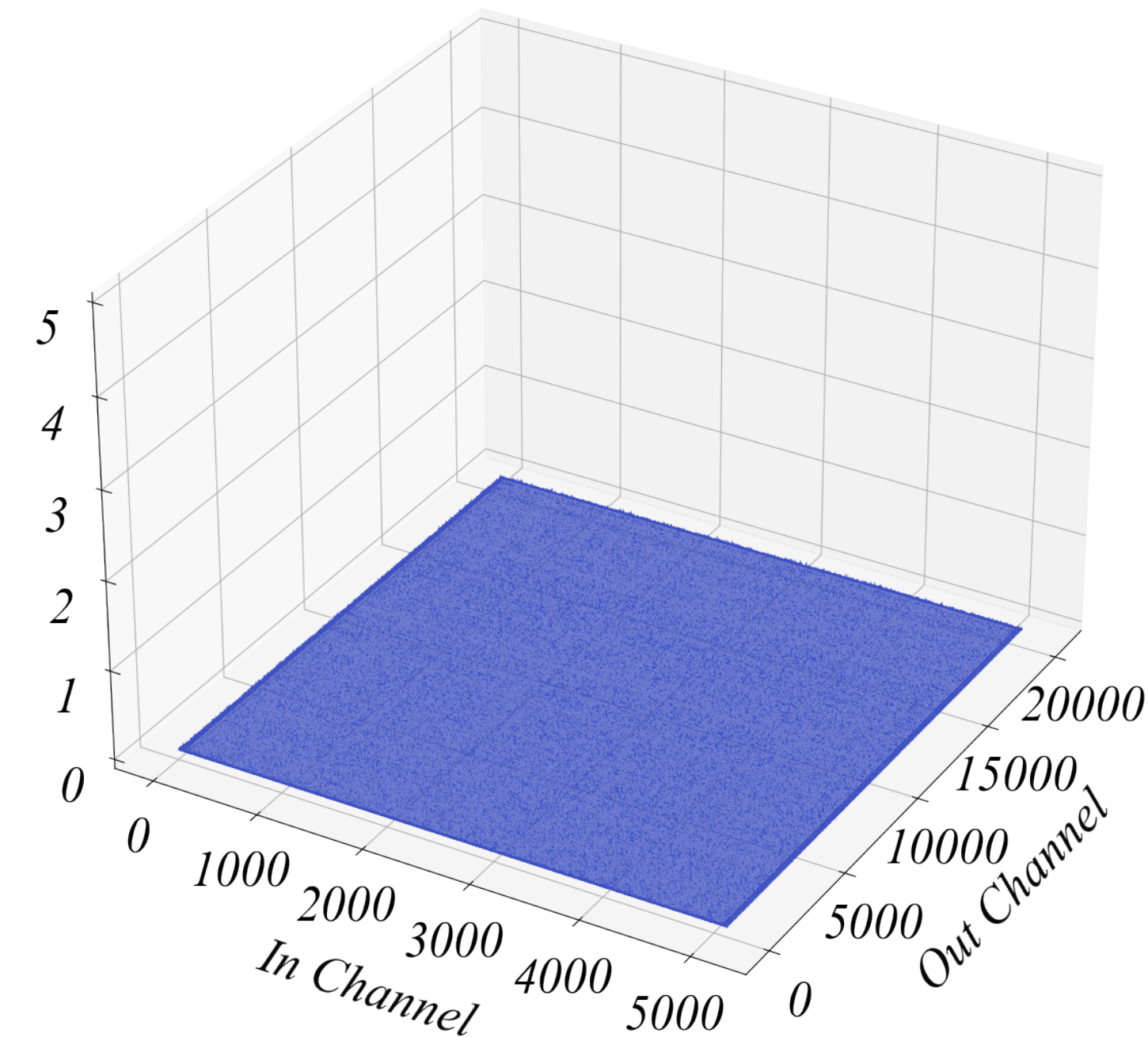
Review the Quantization Difficulty of LLMs

- Activations are harder to quantize than weights
Previous work has shown quantizing the weights of LLMs with INT8 or even INT4 doesn't degrade accuracy.



Activation (Original)

Hard to quantize



Weight (Original)

Very easy to quantize

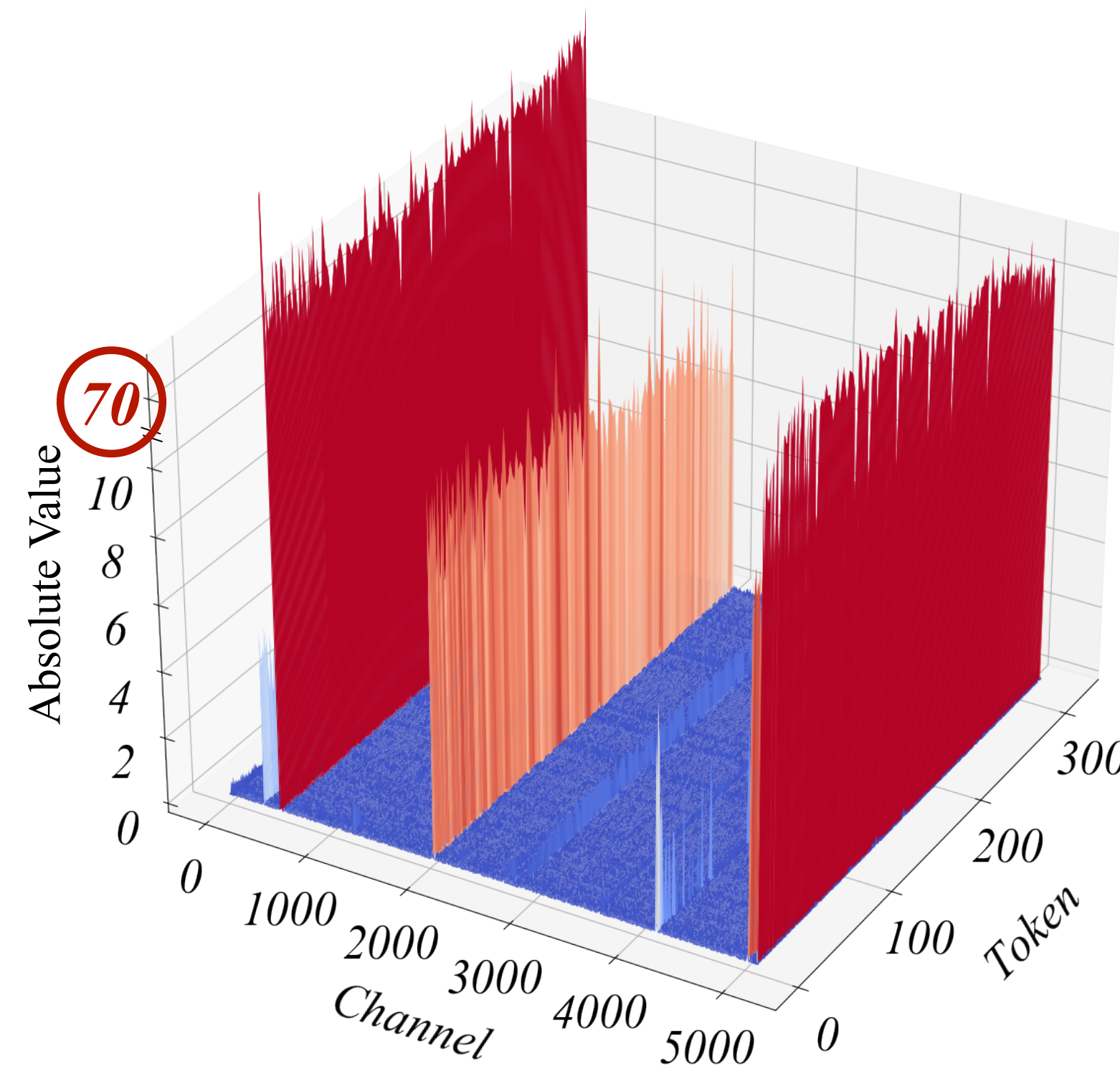
LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale (Dettmers *et al.*, 2022)
GLM-130b: An open bilingual pre-trained model (Zeng *et al.*, 2022)

Review the Quantization Difficulty of LLMs

- Outliers make activation quantization difficult

The scale of outliers is $\sim 100\times$ larger than most of the activation values.

If we use INT8 quantization, most values will be zeroed out.

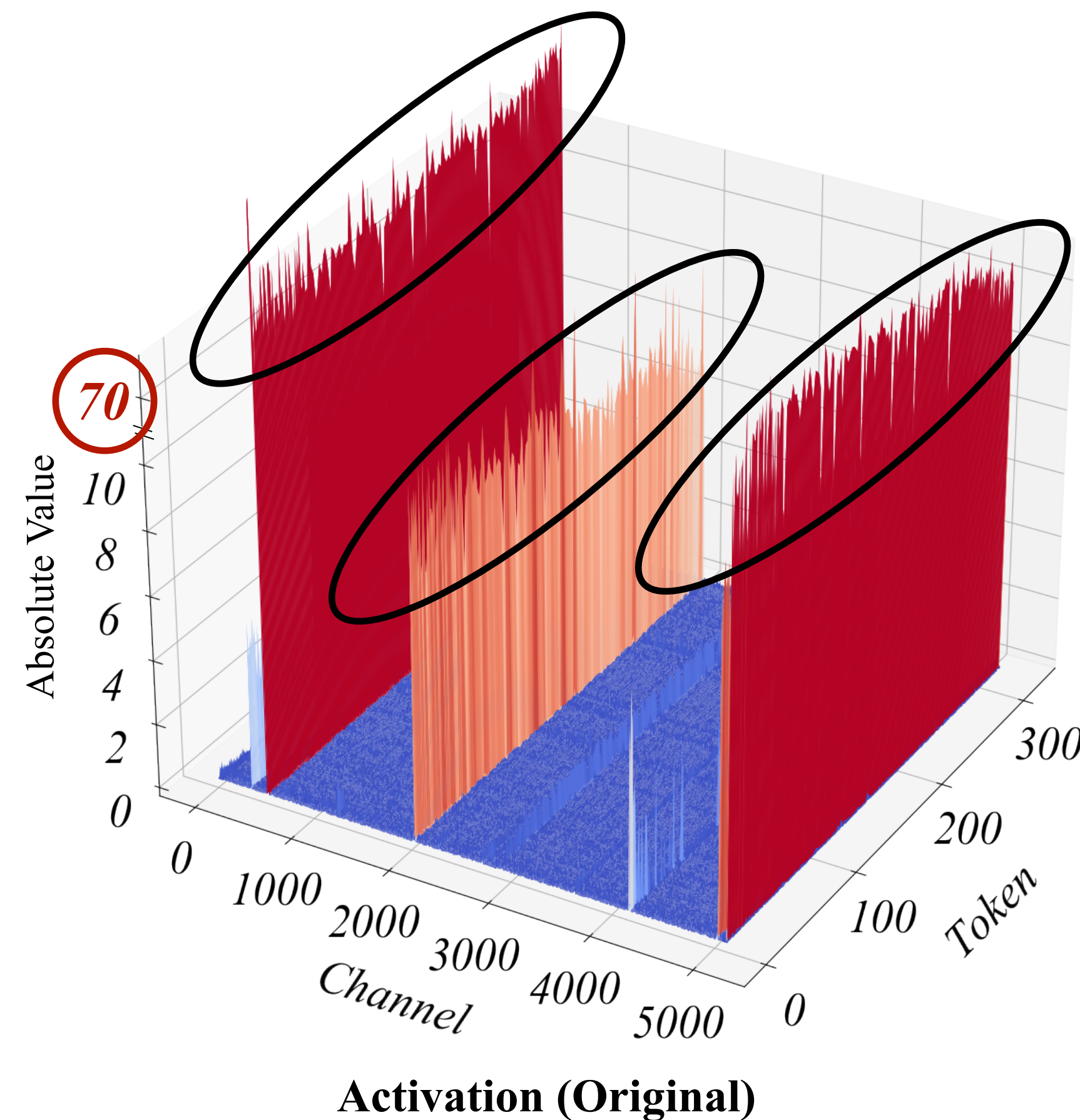


Activation (Original)

Hard to quantize

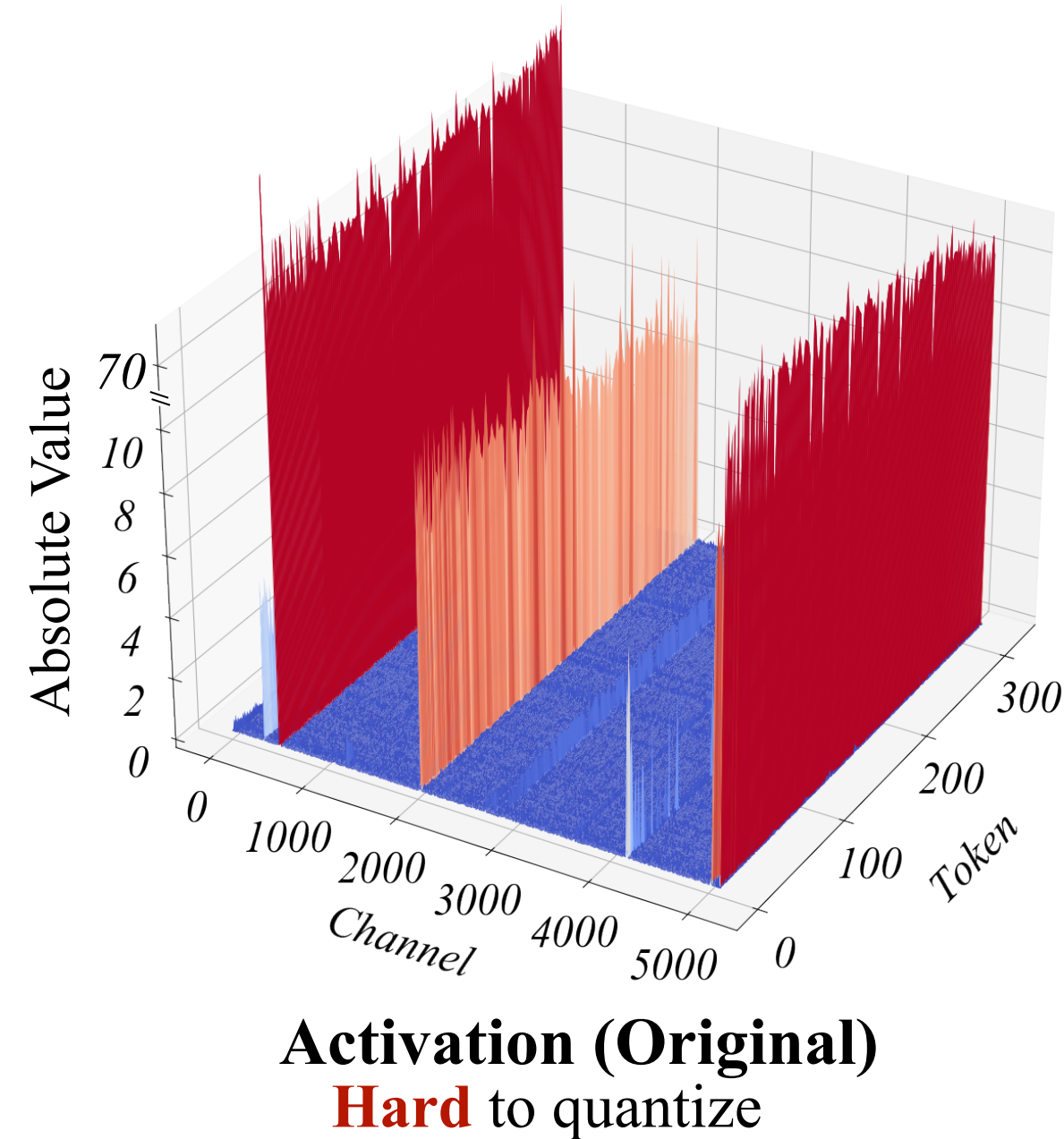
Review the Quantization Difficulty of LLMs

- Outliers persist in *fixed* channels
Fixed channels have outliers, and the outlier channels are persistently large.



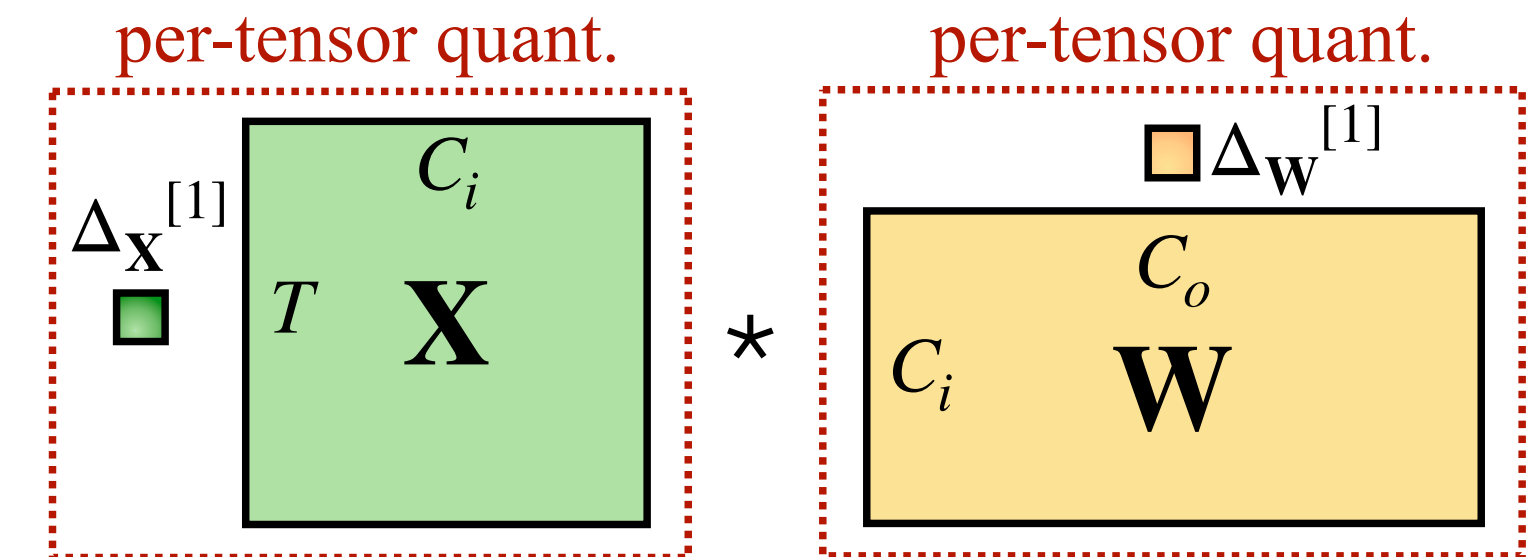
Hard to quantize

Quantization Schemes

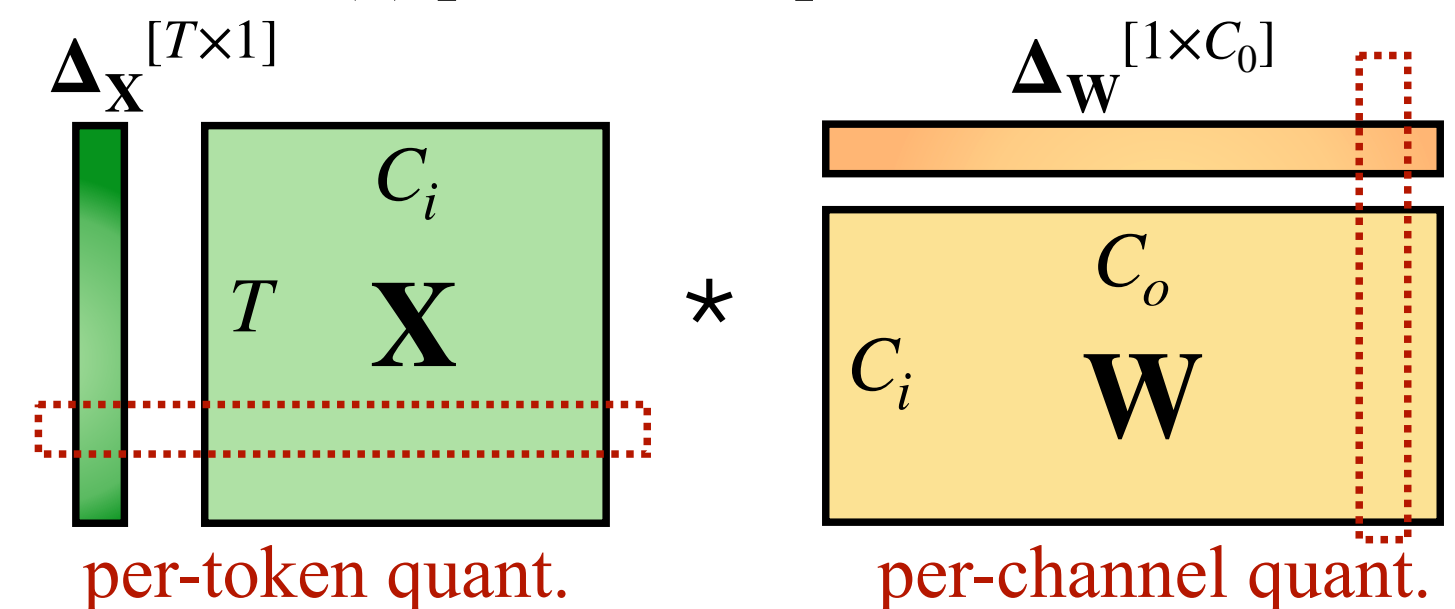


Model size	6.7B	13B	30B	66B	175B
FP16	64.9%	65.6%	67.9%	69.5%	71.6%
INT8 per-tensor	39.9%	33.0%	32.8%	33.1%	32.3%
INT8 per-token	42.5%	33.0%	33.1%	32.9%	31.7%
INT8 per-channel	64.8%	65.6%	68.0%	69.4%	71.4%

Among different activation quantization schemes, only per-channel quantization preserves the accuracy, but it is not compatible with INT8 GEMM kernels.



(a) per-tensor quantization

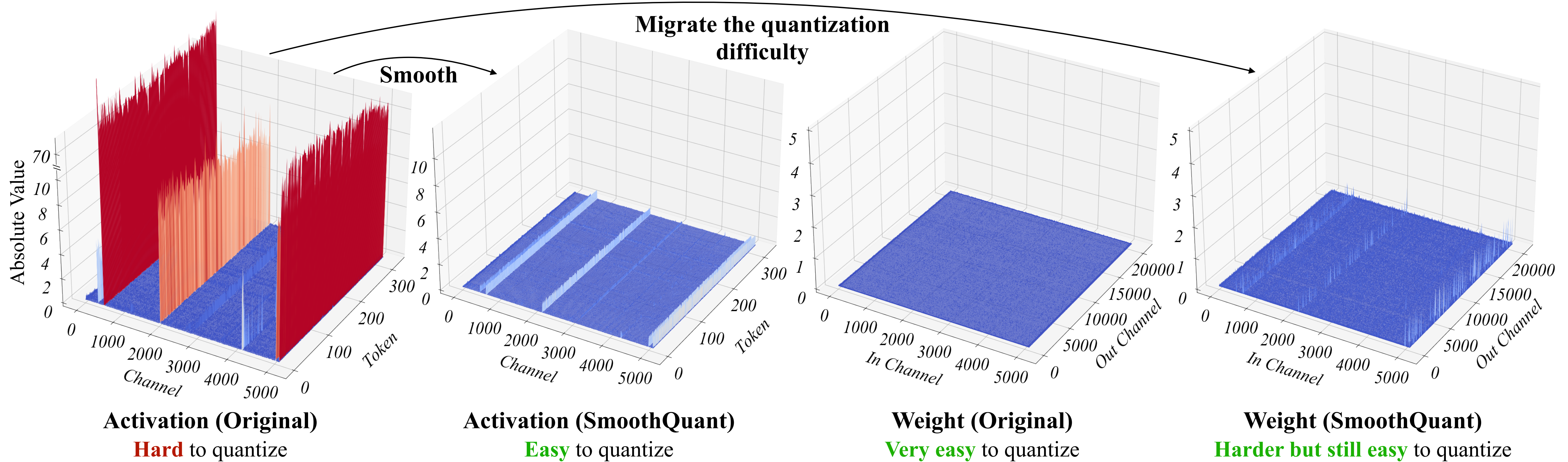


(b) per-token + per-channel quantization

$$\bar{\mathbf{X}}^{\text{INT8}} = \left\lceil \frac{\mathbf{X}^{\text{FP16}}}{\Delta} \right\rceil, \quad \Delta = \frac{\max(|\mathbf{X}|)}{2^{N-1} - 1}$$

$$\mathbf{Y} = \text{diag}(\Delta_{\mathbf{X}}^{\text{FP16}}) \cdot (\bar{\mathbf{X}}^{\text{INT8}} \cdot \bar{\mathbf{W}}^{\text{INT8}}) \cdot \text{diag}(\Delta_{\mathbf{W}}^{\text{FP16}})$$

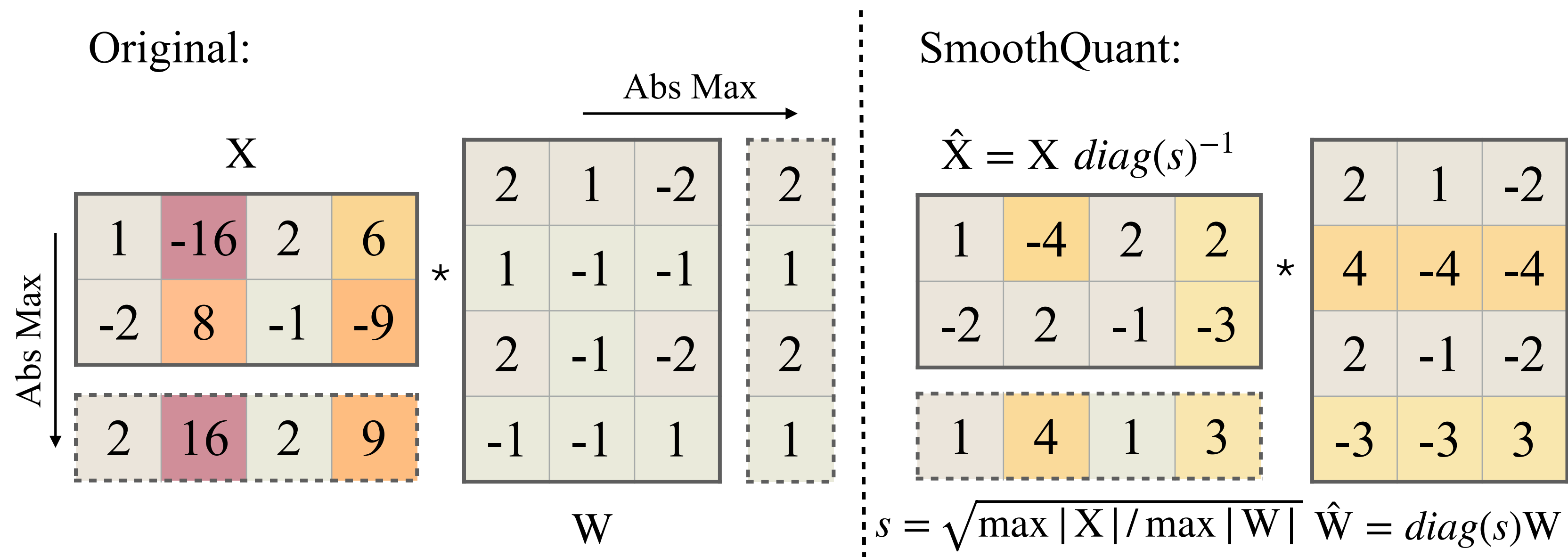
Review the Quantization Difficulty of LLMs



- Activations are harder to quantize than weights
- Outliers make activation quantization difficult
- Outliers persist in *fixed* channels

➡ We can smooth the outlier channels in activations by migrating their magnitudes into the following weights!

Activation Smoothing



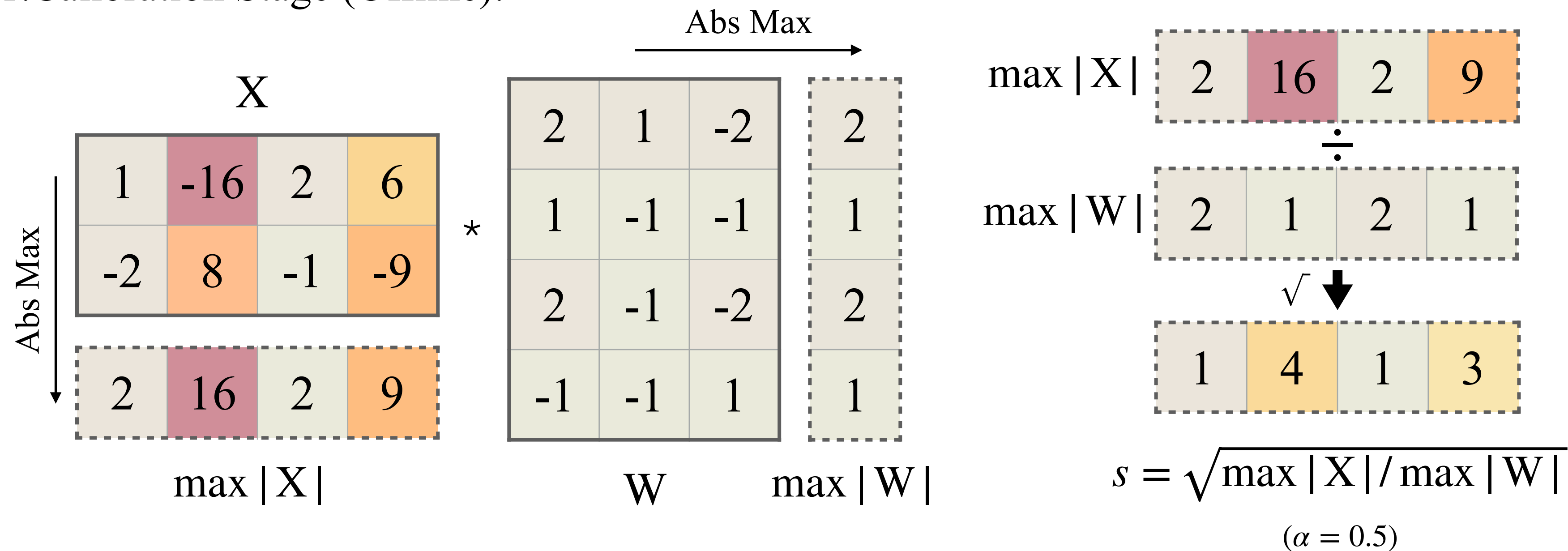
$$s_j = \max(|\mathbf{X}_j|)^\alpha / \max(|\mathbf{W}_j|)^{1-\alpha}, j = 1, 2, \dots, C_i$$

$$\mathbf{Y} = (\mathbf{X} \text{diag}(\mathbf{s})^{-1}) \cdot (\text{diag}(\mathbf{s})\mathbf{W}) = \hat{\mathbf{X}}\hat{\mathbf{W}}$$

α : Migration Strength

Activation Smoothing

1. Calibration Stage (Offline):



$$s_j = \max(|\mathbf{X}_j|)^\alpha / \max(|\mathbf{W}_j|)^{1-\alpha}, j = 1, 2, \dots, C_i$$

α : Migration Strength

Activation Smoothing

2. Smoothing Stage (Offline):

multiply the input channel of the following weight by s

$$\begin{array}{c}
 \mathbf{X} \\
 \begin{array}{|c|c|c|c|}
 \hline
 1 & -16 & 2 & 6 \\
 \hline
 -2 & 8 & -1 & -9 \\
 \hline
 \end{array} \\
 \div \\
 \mathbf{s} \\
 \begin{array}{|c|c|c|c|}
 \hline
 1 & 4 & 1 & 3 \\
 \hline
 \end{array}
 \end{array}
 =
 \begin{array}{c}
 \hat{\mathbf{X}} = \mathbf{X} \mathit{diag}(\mathbf{s})^{-1} \\
 \begin{array}{|c|c|c|c|}
 \hline
 1 & -4 & 2 & 2 \\
 \hline
 -2 & 2 & -1 & -3 \\
 \hline
 \end{array} \\
 \text{divide the output channel} \\
 \text{of the previous layer by s}
 \end{array}
 \times
 \begin{array}{c}
 \mathbf{W} \\
 \begin{array}{|c|c|c|}
 \hline
 2 & 1 & -2 \\
 \hline
 1 & -1 & -1 \\
 \hline
 2 & -1 & -2 \\
 \hline
 -1 & -1 & 1 \\
 \hline
 \end{array} \\
 \mathbf{s} \\
 \begin{array}{|c|}
 \hline
 1 \\
 \hline
 4 \\
 \hline
 1 \\
 \hline
 3 \\
 \hline
 \end{array} \\
 \hat{\mathbf{W}} = \mathit{diag}(\mathbf{s})\mathbf{W} \\
 \begin{array}{|c|c|c|}
 \hline
 2 & 1 & -2 \\
 \hline
 4 & -4 & -4 \\
 \hline
 2 & -1 & -2 \\
 \hline
 -3 & -3 & 3 \\
 \hline
 \end{array}
 \end{array}$$

$$s_j = \max(|\mathbf{X}_j|)^\alpha / \max(|\mathbf{W}_j|)^{1-\alpha}, j = 1, 2, \dots, C_i$$

$$\mathbf{Y} = (\mathbf{X} \mathit{diag}(\mathbf{s})^{-1}) \cdot (\mathit{diag}(\mathbf{s})\mathbf{W}) = \hat{\mathbf{X}}\hat{\mathbf{W}}$$

α : Migration Strength

Activation Smoothing

3. Inference (deployed model):

$$\hat{\mathbf{X}} * \hat{\mathbf{W}}$$

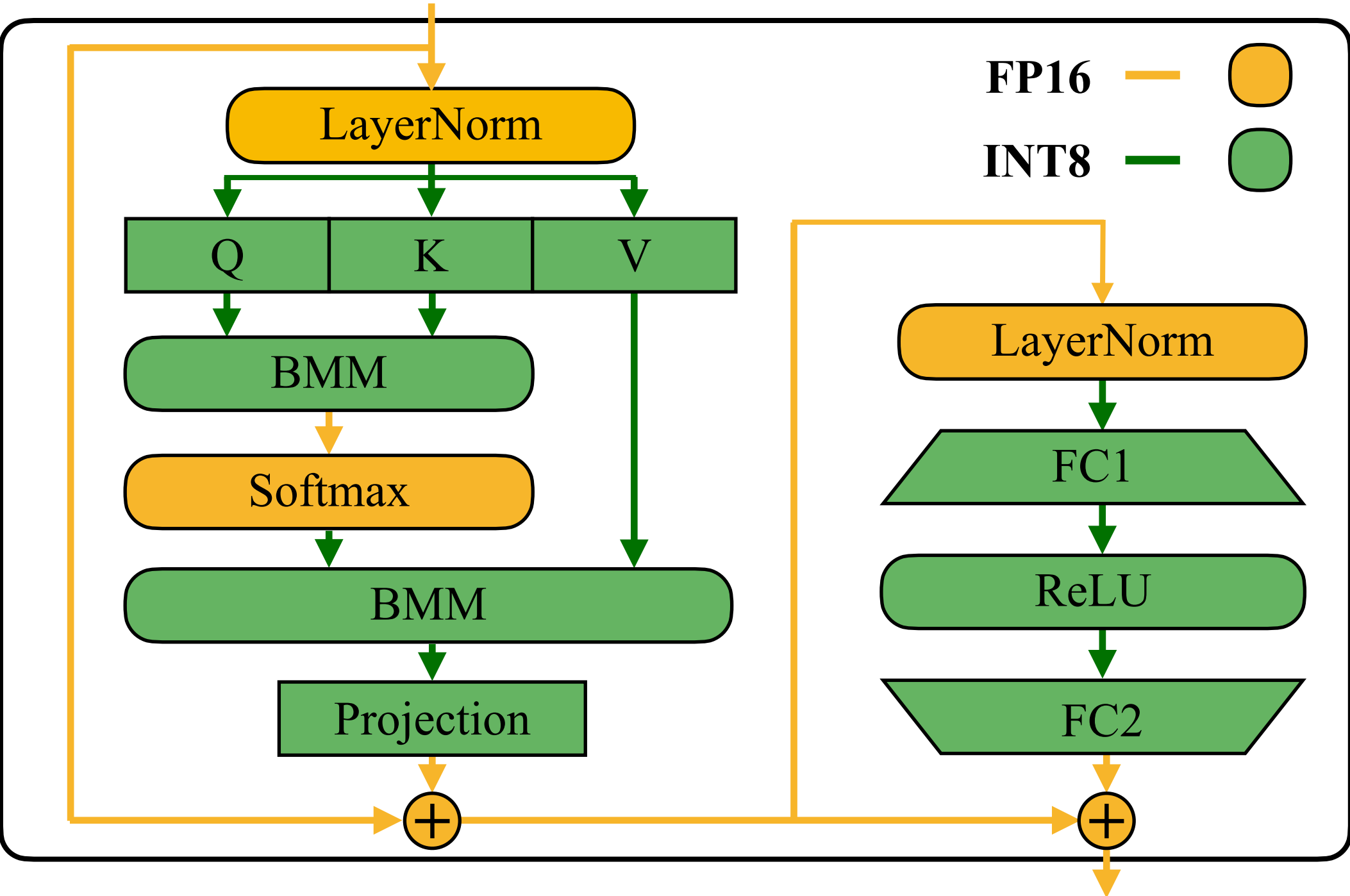
1	-4	2	2
-2	2	-1	-3

2	1	-2
4	-4	-4
2	-1	-2
-3	-3	3

At runtime, the activations are smooth and easy to quantize

$$\mathbf{Y} = \hat{\mathbf{X}}\hat{\mathbf{W}}$$

System Implementation



Method	Weight	Activation
W8A8	per-tensor	per-tensor dynamic
ZeroQuant	group-wise	per-token dynamic
LLM.int8()	per-channel	per-token dynamic+FP16
Outlier Suppression	per-tensor	per-tensor static
SmoothQuant-O1	per-tensor	per-token dynamic
SmoothQuant-O2	per-tensor	per-tensor dynamic
SmoothQuant-O3	per-tensor	per-tensor static

- SmoothQuant’s precision mapping for a Transformer block.
- All compute-intensive operators, such as linear layers and batched matrix multiplications (BMMs) use INT8 arithmetic.
- Quantization setting of the baselines and SmoothQuant. All weight and activations use INT8 representations unless specified.
- We implement three efficiency levels of quantization settings for SmoothQuant. The efficiency improves from O1 to O3.

Accuracy on OPT-175B

<i>OPT-175B</i>	LAMBADA	HellaSwag	PIQA	WinoGrande	OpenBookQA	RTE	COPA	Average↑	WikiText↓
FP16	74.7%	59.3%	79.7%	72.6%	34.0%	59.9%	88.0%	66.9%	10.99
W8A8	0.0%	25.6%	53.4%	50.3%	14.0%	49.5%	56.0%	35.5%	93080
ZeroQuant	0.0%*	26.0%	51.7%	49.3%	17.8%	50.9%	55.0%	35.8%	84648
LLM.int8()	74.7%	59.2%	79.7%	72.1%	34.2%	60.3%	87.0%	66.7%	11.10
Outlier Suppression	0.00%	25.8%	52.5%	48.6%	16.6%	53.4%	55.0%	36.0%	96151
SmoothQuant-O1	74.7%	59.2%	79.7%	71.2%	33.4%	58.1%	89.0%	66.5%	11.11
SmoothQuant-O2	75.0%	59.0%	79.2%	71.2%	33.0%	59.6%	88.0%	66.4%	11.14
SmoothQuant-O3	74.6%	58.9%	79.7%	71.2%	33.4%	59.9%	90.0%	66.8%	11.17

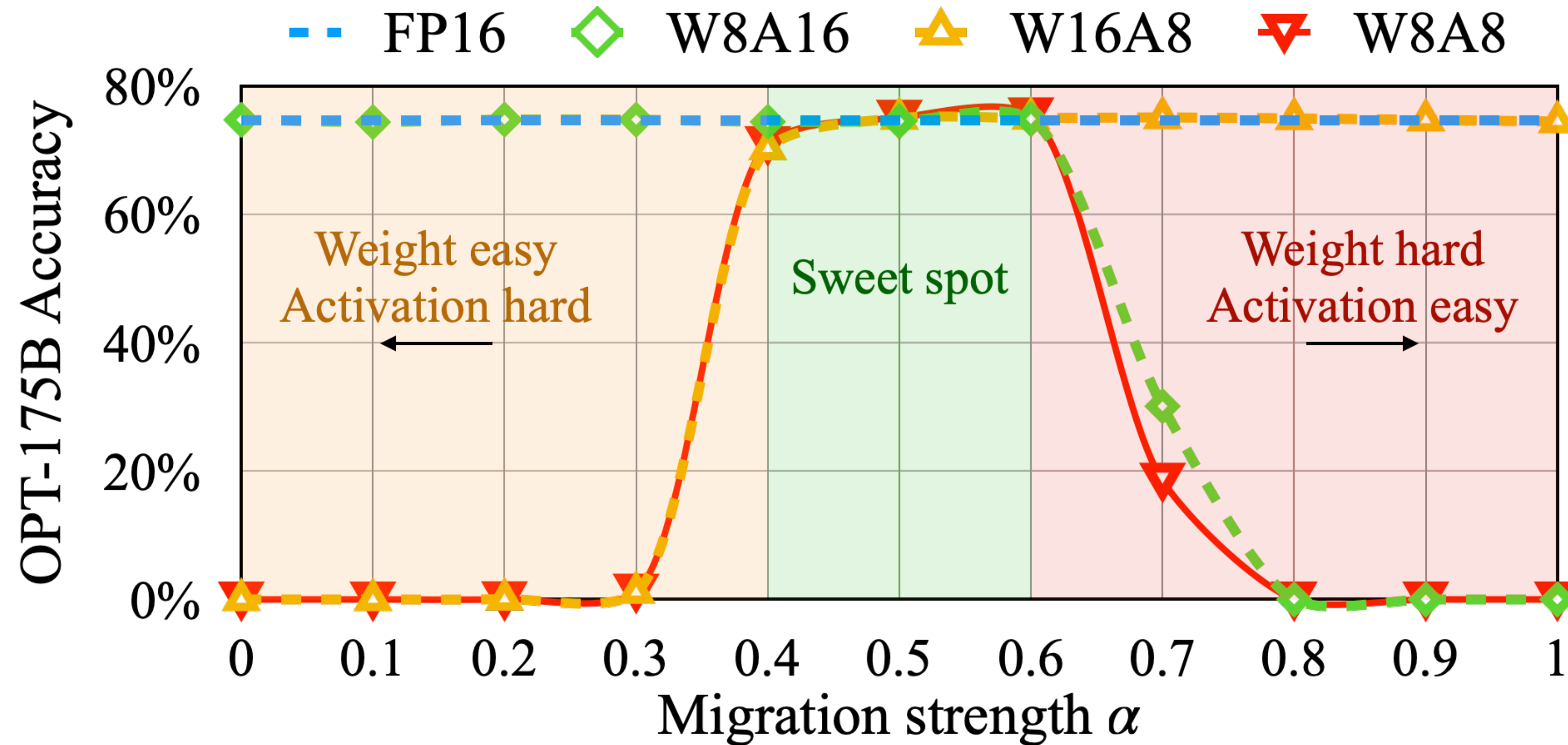
SmoothQuant maintains the accuracy of OPT-175B model after INT8 quantization, even with the most aggressive and most efficient O3 setting.

Accuracy on Different LLMs

Method	OPT-175B	BLOOM-176B	GLM-130B*
FP16	71.6%	68.2%	73.8%
W8A8	32.3%	64.2%	26.9%
ZeroQuant	31.7%	67.4%	26.7%
LLM.int8()	71.4%	68.0%	73.8%
Outlier Suppression	31.7%	54.1%	63.5%
SmoothQuant-O1	71.2%	68.3%	73.7%
SmoothQuant-O2	71.1%	68.4%	72.5%
SmoothQuant-O3	71.1%	67.4%	72.8%

SmoothQuant works for different LLMs. We can quantize the 3 largest, openly available LLM models into INT8 without degrading the accuracy.

Ablation Study on the Migration Strength α

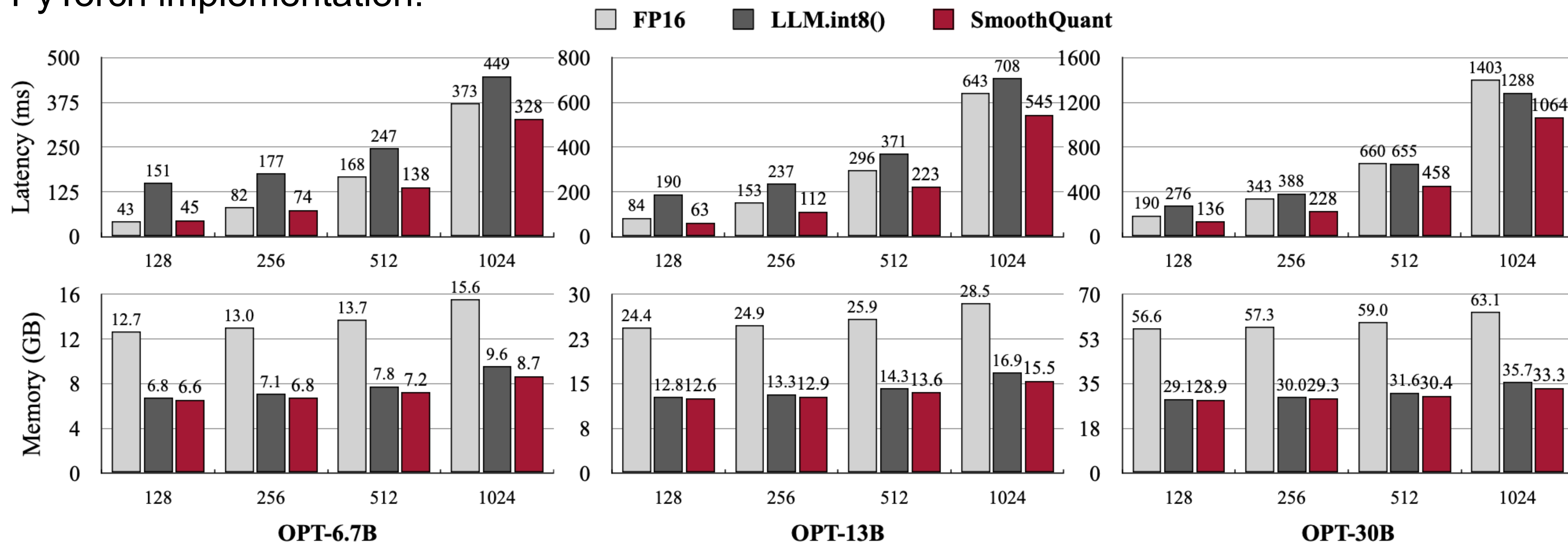


$$s_j = \max(|\mathbf{X}_j|)^\alpha / \max(|\mathbf{W}_j|)^{1-\alpha}, j = 1, 2, \dots, C_i \quad \mathbf{Y} = (\mathbf{X} \text{diag}(\mathbf{s})^{-1}) \cdot (\text{diag}(\mathbf{s})\mathbf{W}) = \hat{\mathbf{X}}\hat{\mathbf{W}}$$

- Migration strength α controls the amount of quantization difficulty migrated from activations to weights.
- A suitable migration strength α (sweet spot) makes both activations and weights easy to quantize.
- If the α is too large, weights will be hard to quantize; if too small, activations will be hard to quantize.

Speedup and Memory Saving

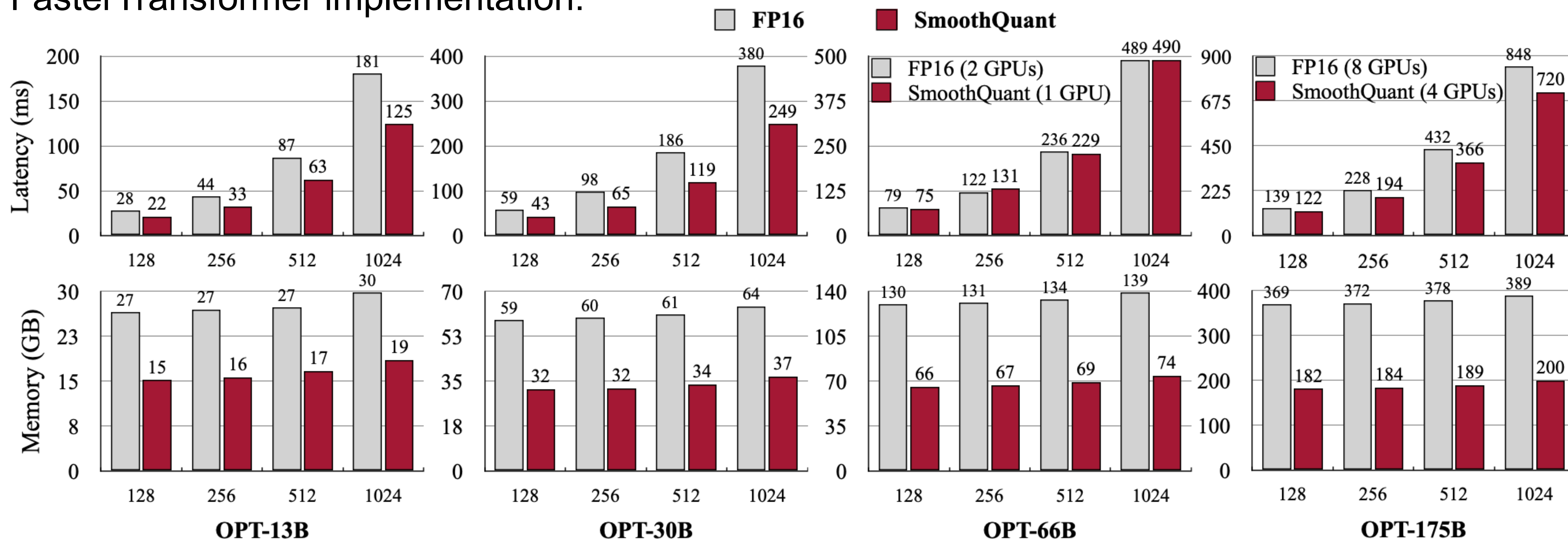
PyTorch implementation:



The PyTorch implementation of SmoothQuant achieves up to **1.51x** speedup and **1.96x** memory saving for OPT models on a single NVIDIA A100-80GB GPU, while **LLM.int8()** slows down the inference in most cases.

Speedup and Memory Saving

FasterTransformer implementation:



- We integrate SmoothQuant into FasterTransformer, a state-of-the-art Transformer serving framework.
- For smaller models, the latency can be significantly reduced with SmoothQuant by up to **1.56x** compared to FP16.
- For the bigger models (OPT-66B and 175B), we can achieve similar or even faster inference using only **half** number of GPUs. Memory footprint is almost halved compared to FP16.

Conclusion

- We propose SmoothQuant, a turn-key solution to enable accurate W8A8 quantization for large language models.
- SmoothQuant is accurate and efficient on existing hardware. We can implement SmoothQuant with off-the-shelf kernels to achieve high speedup and memory saving.

- Paper: <https://arxiv.org/abs/2211.10438>
- Code: <https://github.com/mit-han-lab/smoothquant>